

Full Stack Development with MERN

Project Documentation format

1. Introduction

- **Project Title:** House Rent
- **Team Members:**
 1. Bala Swathi Kapa
 2. Devika Sandadi
 3. Lakshmi Gayathri Padala
 4. Peeda Mahalakshmi Kalvakolanu
 5. Ramyasri K

2. Project Overview

- **Purpose:**

The House Rent project is a web-based application developed using the MERN Stack (MongoDB, Express.js, React.js, and Node.js).

The main purpose of this project is to provide an online platform where users can easily search, view, and book rental properties. Users can register, log in securely, browse available houses, search properties by title or location, view property details, and book houses through a simple and user-friendly interface.

The application reduces the manual effort involved in house renting and provides a faster, more convenient, and transparent rental process.

- **Features:**

The **House Rent** application provides the following key features:

User Registration: New users can create an account by entering their name, email, password, and other required details. The registration process allows users to securely access the application.

Secure Login: Registered users can log in using their email and password. Authentication is implemented using **JSON Web Token (JWT)**, ensuring that only authorized users can access protected pages.

Browse Properties: Users can explore a list of rental properties available in different locations. Each property displays basic details such as title, description, location, and monthly rent.

Search Properties: The application provides a search feature that allows users to search properties based on the property title or location, making it easier to find suitable rental houses.

Property Details: Users can view complete information about a selected property, including description, location, rental price, and property image before proceeding with booking.

Property Booking: Authenticated users can book a rental property directly through the application. The booking details are stored securely in the database for future reference.

User Profile Management: Users can view and manage their profile information after successful login. This includes details such as name, email, and account role.

Admin Panel: The application includes an admin module where administrators can manage property listings by adding, updating, or deleting property information.

Responsive User Interface: The frontend is developed using React.js with a responsive design, allowing the application to work efficiently on desktops, laptops, tablets, and mobile devices.

MongoDB Atlas Integration: All application data, including user accounts, property information, and booking details, is securely stored in **MongoDB Atlas**, ensuring reliable cloud-based data management.

RESTful APIs: The frontend communicates with the backend through RESTful APIs developed using Express.js. These APIs handle user authentication, property management, and booking operations.

Data Security: User passwords are encrypted using **bcrypt.js**, while **JSON Web Tokens (JWT)** are used for authentication and authorization, ensuring secure access to protected resources.

Fast Navigation: The application uses **React Router DOM** to provide seamless navigation between different pages without refreshing the browser, resulting in a smooth user experience.

3. Architecture

- **Frontend (React.js):**

The frontend of the **House Rent** application is developed using **React.js**, which provides a responsive and interactive user interface. The application is built using reusable React components, making the code modular and easy to maintain. **React Router DOM** is used for smooth navigation between pages without refreshing the browser, while **Axios** is used to communicate with the backend APIs.

The frontend includes pages such as **Home, Register, Login, Properties, Property Details, Booking, My Bookings, Profile, and Admin**. Users can browse available houses, search properties, view detailed information, and book rental properties through an easy-to-use interface.

- **Backend (Node.js & Express.js):**

The backend is developed using **Node.js** and **Express.js**, which handle server-side operations and API management. It follows a modular structure with **Models, Controllers, Routes, and Middleware**.

The backend provides RESTful APIs for **user authentication, property management, profile management, and booking operations**. User passwords are encrypted using **bcrypt.js**, while **JSON Web Tokens (JWT)** are used for secure authentication and authorization of users.

- **Database (MongoDB Atlas):**

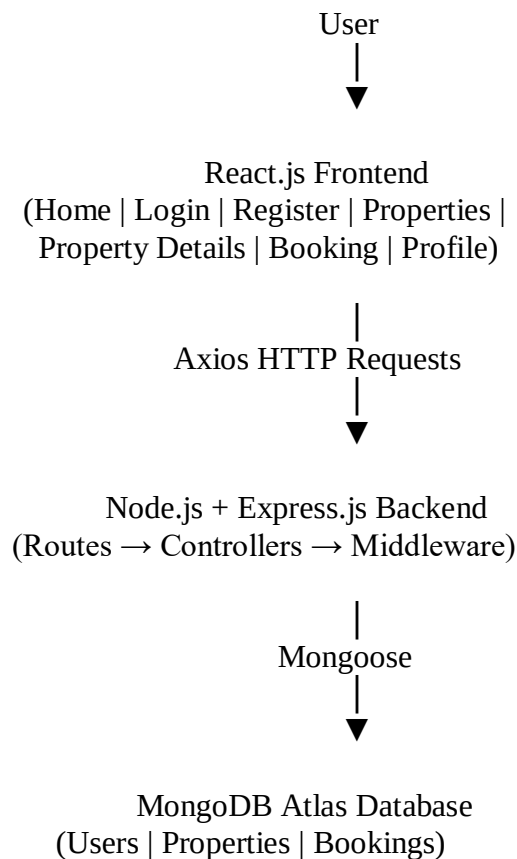
The application uses **MongoDB Atlas**, a cloud-based NoSQL database, to store and manage application data. **Mongoose** is used as the ODM library for defining schemas and performing database operations.

The database mainly contains the following collections:

- **Users** – Stores user information such as name, email, encrypted password, and role.
- **Properties** – Stores property details including title, description, location, price, and other information.
- **Bookings** – Stores booking details of users for selected rental properties.

MongoDB Atlas ensures secure cloud storage, efficient data retrieval, and reliable application performance.

- **Architecture Workflow:**



4. Setup Instructions

- **Prerequisites:**

Before running the **House Rent** application, ensure that the following software and tools are installed on your system.

Software	Version (Recommended)	Purpose
Node.js	v18 or later	JavaScript Runtime Environment
npm	Latest Version	Package Manager
MongoDB Atlas	Cloud Database	Store Application Data
Visual Studio Code	Latest Version	Code Editor
Git	Latest Version	Version Control
GitHub	Latest Version	Source Code Repository
Postman	Latest Version	API Testing
Google Chrome	Latest Version	Web Browser

1.Frontend Dependencies

The frontend of the **House Rent** application is developed using **React.js** and requires the following packages:

- **React.js** – Used to build a responsive and interactive user interface.
- **React Router DOM** – Enables seamless navigation between different pages without reloading the application.
- **Axios** – Used to send HTTP requests from the frontend to the backend APIs.
- **CSS** – Used for designing and styling the user interface.
- **React DOM** – Renders React components in the browser.

2.Backend Dependencies

The backend is developed using **Node.js** and **Express.js** with the following packages:

- **Express.js** – Provides routing and API development.
- **Mongoose** – Connects the application with MongoDB Atlas and performs database operations.
- **bcrypt.js** – Encrypts user passwords before storing them in the database.
- **JSON Web Token (JWT)** – Provides secure user authentication and authorization.
- **dotenv** – Manages environment variables securely.

- **cors** – Allows communication between the frontend and backend running on different ports.
- **nodemon** – Automatically restarts the server whenever backend code changes during development.

These dependencies help in developing a secure, scalable, and efficient House Rent application by supporting authentication, database connectivity, API communication, and server management.

Installation:

Step 1: Clone the Repository

Clone the project repository from GitHub.

```
git clone https://github.com/balaswathikapa-gif/House-Rent.git
```

Move into the project directory.

```
cd House-Rent
```

Step 2: Install Frontend Dependencies

Navigate to the client folder.

```
cd client
```

Install all required packages.

```
npm install
```

Step 3: Install Backend Dependencies

Open another terminal and navigate to the server folder.

```
cd server
```

Install backend packages.

```
npm install
```

Step 4: Configure Environment Variables

Create a **.env** file inside the **server** folder and configure the required environment variables.

```
PORT=5000
```

```
MONGO_URI=Your_MongoDB_Connection_String
```

```
JWT_SECRET=Your_Secret_Key
```

Step 5: Start the Backend Server

Run the backend server.

```
cd server
```

```
npm run dev
```

The backend server will start on:

```
http://localhost:5000
```

MongoDB Connected Successfully

Step 6: Start the Frontend Application

Open another terminal.

Navigate to the client folder.

cd client

Start the React application.

npm run dev

The application will be available at:

http://localhost:5173

Step 7: Verify the Application

Open the application in your web browser.

Verify that the following modules are working correctly:

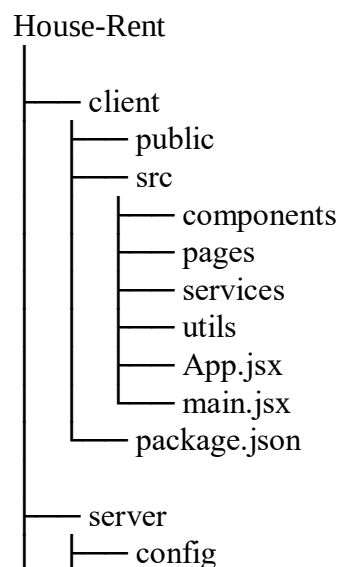
- Home Page
- User Registration
- User Login
- Properties Page
- Property Details
- Property Booking
- My Bookings
- User Profile
- Admin Panel

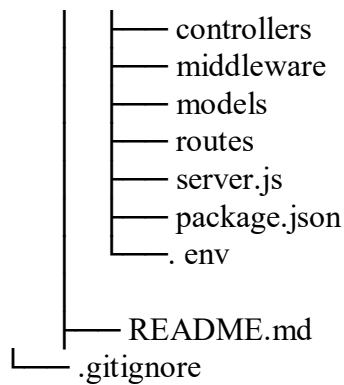
If all the pages load successfully, user authentication works correctly, and the backend is connected to **MongoDB Atlas**, the installation is completed successfully.

Environment Variables Used

Variable	Description
PORT	Specifies the backend server port number
MONGO_URI	MongoDB Atlas database connection string
JWT_SECRET	Secret key used for generating JWT authentication tokens

• Project Structure After Installation





5. Folder Structure

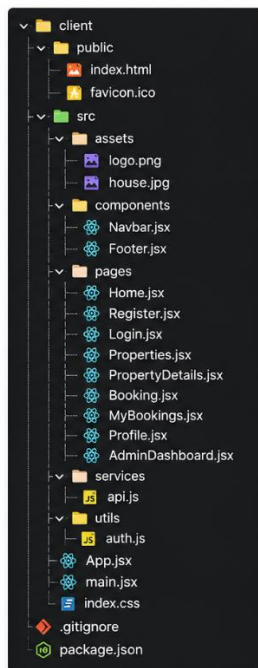
The House Rent project follows a modular folder structure that separates the frontend and backend into different directories. This organization improves code readability, simplifies maintenance, and makes future enhancements easier.

- **Client:**

The **client** folder contains the complete frontend of the **House Rent** application. It is developed using **React.js** and is responsible for displaying the user interface, handling user interactions, and communicating with the backend through REST APIs.

- **Description of Client Folder**

Folder/File	Description
public	Stores static resources such as images and other public files.
components	Contains reusable UI components such as Navbar and Footer.
pages	Includes application pages such as Home, Register, Login, Properties, Property Details, Booking, My Bookings, Profile, and Admin.
services	Contains Axios configuration for communicating with backend APIs.
App.jsx	Main component that defines application routes using React Router.
main.jsx	Entry point of the React application.
utils	Stores helper functions related to authentication and common utilities.



Client Folder Structure

Folder / File	Description
public	Contains the static files like index.html and favicon.
src	Contains the source code of the React application.
assets	Stores images and other static resources used in the application.
components	Contains reusable UI components such as Navbar and Footer.
pages	Contains all the pages of the application.
Home.jsx	Home page of the application.
Register.jsx	User registration page.
Login.jsx	User login page.
Properties.jsx	Displays list of all available properties.
PropertyDetails.jsx	Displays detailed information of a selected property.
Booking.jsx	Page for booking a property.
MyBookings.jsx	Displays bookings made by the logged-in user.
Profile.jsx	User profile page.
AdminDashboard.jsx	Admin panel for managing properties and bookings.
services	Contains Axios configuration and API calls.
api.js	Handles API requests to the backend.
utils	Contains utility/helper functions.
auth.js	Handles authentication related functions.
App.jsx	Main component that defines all the application routes.
main.jsx	Entry point of the React application.
index.css	Contains global styles for the application.
.gitignore	Specifies files and folders to be ignored by Git.
package.json	Contains project metadata and dependencies.

The frontend is designed to provide a responsive and user-friendly experience while ensuring smooth navigation between different pages.

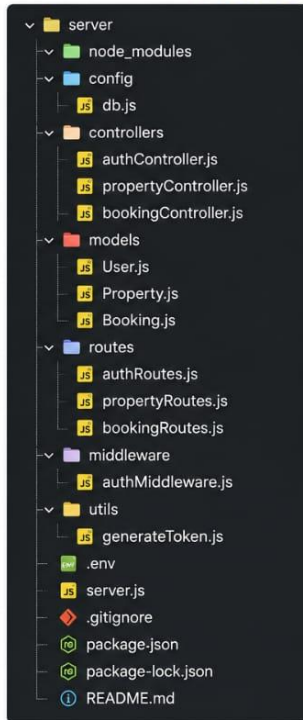
- **Server:**

The **server** folder contains the backend of the **House Rent** application. It is developed using **Node.js** and **Express.js**. The backend manages business logic, authentication, API requests, and database communication.

- **Description of Server Folder**

Folder/File	Description
controllers	Contains business logic for authentication, property management, bookings, and profile operations.
middleware	Stores JWT authentication and authorization middleware.
models	Defines MongoDB schemas using Mongoose for Users, Properties, and Bookings.
config	Contains MongoDB database connection configuration.
routes	Defines REST API endpoints and connects them with the appropriate controllers.
.env	Stores environment variables such as MongoDB connection string and JWT secret key.
server.js	Main entry point of the backend application. It connects to MongoDB, configures middleware, and starts the Express server.

SERVER FOLDER STRUCTURE – House Rent Management System



Folder / File	Purpose
node_modules/	Contains all installed dependencies.
config/	Configuration files for database and environment.
db.js	MongoDB connection setup.
controllers/	Contains all controller files (business logic).
authController.js	Handles user registration and login logic.
propertyController.js	Handles property related operations.
bookingController.js	Handles booking related operations.
models/	Contains Mongoose schema and models.
User.js	User schema and model.
Property.js	Property schema and model.
Booking.js	Booking schema and model.
routes/	Contains all route files.
authRoutes.js	Authentication related API routes.
propertyRoutes.js	Property related API routes.
bookingRoutes.js	Booking related API routes.
middleware/	Custom middleware functions.
authMiddleware.js	Protects routes and verifies JWT token.
utils/	Utility/helper functions.
generateToken.js	Generates JWT token.
.env	Stores environment variables (Mongo URI, JWT Secret, PORT, etc.).
server.js	Entry point of the application. Initializes server and routes.
.gitignore	Specifies files/folders to be ignored by Git.
package.json	Project metadata and dependencies.
package-lock.json	Exact version of installed packages.
README.md	Project documentation.

STRUCTURE OVERVIEW

- MVC Pattern Followed
 - Models → models/
 - Views (Frontend) → client/
 - Controllers → controllers/
- Tech Stack
 - Node.js
 - Express.js
 - MongoDB (Mongoose)
 - JWT for Authentication
- Main Features
 - User Authentication
 - Property Management
 - Booking Management

The backend follows the **MVC (Model–View–Controller)** architecture, where models manage database operations, controllers contain application logic, routes define API endpoints, and middleware handles authentication and request processing.

6. Running the Application

The **House Rent Management System** consists of two separate applications:

- **Frontend (React + Vite)**
- **Backend (Node.js + Express.js)**

Both servers must be started to run the complete application.

○ Frontend:

Navigate to the client folder and install the required dependencies (if not already installed).

```
cd client
npm install
```

Start the frontend development server:

```
npm run dev
```

After successful execution, the application will be available at:

<http://localhost:5173>

- **Backend:**

Open another terminal, navigate to the server folder, and install the dependencies.

```
cd server  
npm install
```

Start the backend server:

```
npm run dev
```

The backend server will start on:

<http://localhost:5000>

Running the Complete Application

1. Start the **Backend Server** using **npm run dev**.
2. Start the **Frontend Server** using **npm run dev**.
3. Open the browser and visit:
<http://localhost:5173>
4. Users can now register, log in, browse available rental properties, search properties, view property details, book houses, manage bookings, and access the admin panel.

Note: Since the project is developed using **Vite**, use **npm run dev** instead of **npm start**.

7. API Documentation

The backend of the **House Rent Management System** exposes RESTful APIs for user authentication, property management, booking management, and profile management. All APIs exchange data in **JSON** format.

1. Authentication APIs

Register User

Method	Endpoint	Description
POST	/api/auth/register	Register a new user

Request Body

```
{  
  
  "name": "Bala Swathi",
```

```
"email": "balaswathi@gmail.com",  
"password": "123456"  
}
```

Success Response

```
{  
  "message": "User Registered Successfully"  
}
```

Login User

Method	Endpoint	Description
POST	/api/auth/login	Login user

Request Body

```
{  
  "email": "balaswathi@gmail.com",  
  "password": "123456"  
}
```

Success Response

```
{  
  "token": "JWT_TOKEN"  
}
```

2. Book APIs

Get All Properties

Method	Endpoint	Description
--------	----------	-------------

GET	/api/properties	Retrieve all properties
------------	-----------------	-------------------------

Response

```
[  
  
  {  
  
    "_id": "1",  
  
    "title": "3BHK Villa",  
  
    "location": "Hyderabad",  
  
    "price": 25000  
  
  }  
  
]
```

Get Single Property

Method	Endpoint	Description
GET	/api/books/:id	Retrieve book details

Add New Property (Admin)

Method	Endpoint	Description
POST	/api/properties	Add a new property

Request Body

```
{  
  
  "title": "2BHK Apartment",  
  
  "location": "Vijayawada",  
  
  "price": 18000,  
  
  "description": "Fully Furnished Apartment"  
  
}
```

Update Property

Method	Endpoint	Description
--------	----------	-------------

PUT	/api/properties/:id	Update Property information
------------	---------------------	-----------------------------

Delete Property

Method	Endpoint	Description
DELETE	/api/properties/:id	Delete a Property

3. Booking APIs

Book Property

Method	Endpoint	Description
GET	/api/bookings	Book a rental property

Request Body

```
{  
  "property": "3BHK Villa",  
  "date": "2026-07-20"  
}
```

Success Response

```
{  
  "message": "Booking Successful"  
}
```

Get My Bookings

Method	Endpoint	Description
GET	/api/bookings	View all bookings

Delete Booking

Method	Endpoint	Description
DELETE	/api/bookings/:id	Cancel Booking

4. Protected Routes

The following APIs require JWT Authentication.

- POST /api/properties (*Admin Only*)
- PUT /api/properties/:id (*Admin Only*)
- DELETE /api/properties/:id (*Admin Only*)
- POST /api/bookings
- GET /api/bookings
- DELETE /api/bookings/:id

Authorization Header:

Authorization: Bearer <JWT_TOKEN>

API Summary

Module	Methods
Authentication	Register, Login
Properties	Get All, Add, Update, Delete
Bookings	Book Property, View Bookings, Cancel Booking
Delete Property	Delete a property from the database (Admin only).

This documentation clearly explains all the backend endpoints, their request methods, required parameters, and example responses, making it suitable for your project report.

8. Authentication

- Authentication and Authorization

The House Rent Management System implements JWT (JSON Web Token) based authentication to ensure that only authorized users can access protected resources. User passwords are securely encrypted using the bcrypt.js library before being stored in the MongoDB Atlas database.

- **User Registration**

- A new user registers by providing their **Name, Email, and Password**.
- The password is encrypted using **bcrypt.js** before storing it in the database.
- The user information is securely saved in MongoDB Atlas.

- **User Login**

- The registered user logs in using their **Email** and **Password**.
- The entered password is verified with the encrypted password stored in the database.
- If the credentials are valid, the server generates a **JWT Token** containing the user's information.
- The generated token is returned to the frontend after successful login.

- **JWT Token**

- The JWT token is stored in the browser's **Local Storage**.
- It is sent with every protected API request through the **Authorization Header**.
- **Example**
- `Authorization: Bearer <JWT_TOKEN>`

- **Authorization Process**

- Protected routes are secured using **Authentication Middleware**.
- The middleware verifies the JWT token before processing the request.
- If the token is valid, the request is allowed to access protected resources.
- If the token is missing or invalid, the server returns an **Unauthorized (401)** response.

- **Role-Based Access**

The application supports role-based authorization.

User

- ☐ Register and Login
- ☐ Browse Available Properties

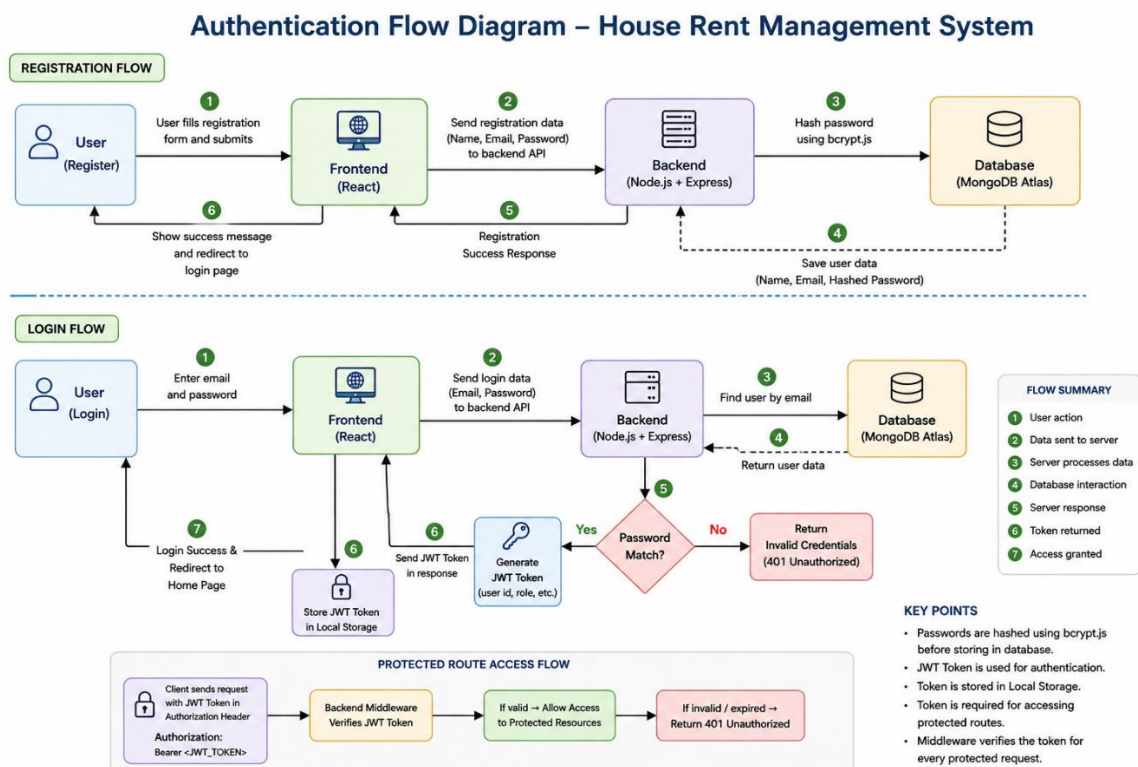
- ☐ Search Properties
- ☐ View Property Details
- ☐ Book Rental Properties
- ☐ View Personal Bookings

Admin

- ☐ Add New Properties
- ☐ Update Property Details
- ☐ Delete Properties
- ☐ Manage Available Property Listings

Security Features

- ☐ Password hashing using **bcrypt.js**
- ☐ JWT-based user authentication
- ☐ Protected API endpoints
- ☐ Authentication middleware for route protection
- ☐ Secure MongoDB Atlas database connectivity
- ☐ Environment variables used to securely store sensitive information such as the JWT Secret Key and MongoDB URI



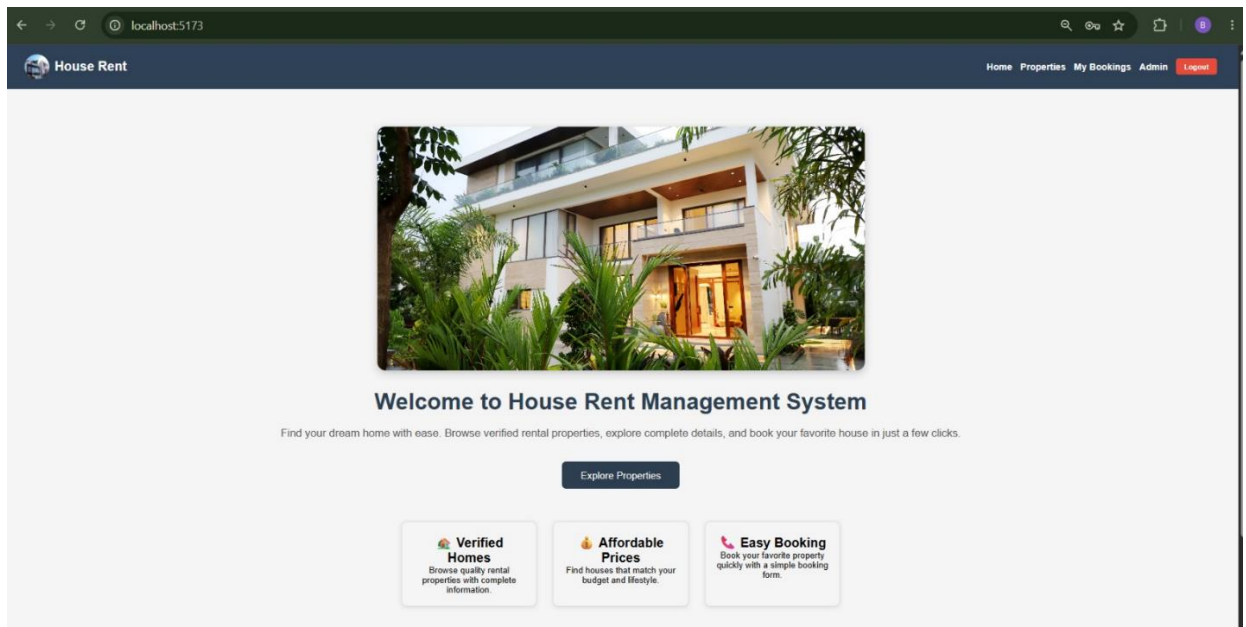
9. User Interface

The **House Rent Management System** provides a clean, responsive, and user-friendly interface developed using **React.js**. The interface is designed to provide smooth navigation and an easy property booking experience across desktops, tablets, and mobile devices.

User Interface Screens

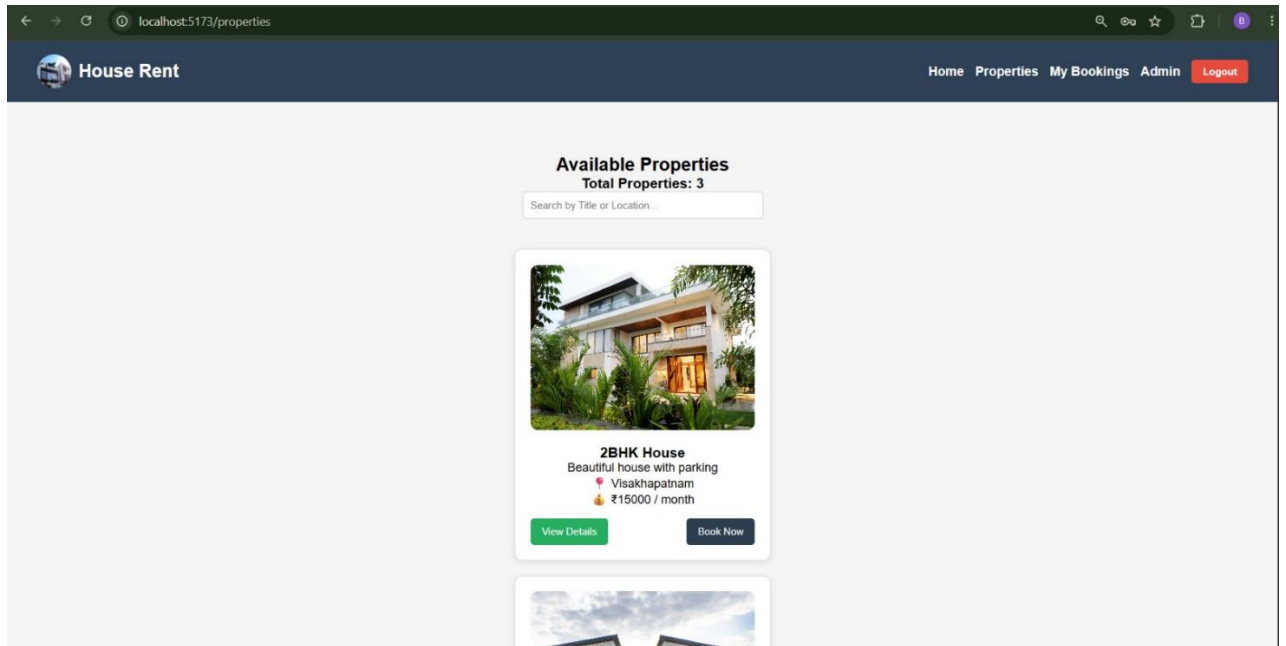
1. Home Page

- Displays the welcome banner and navigation menu.
- Provides quick access to Properties, Login, Register, and Booking pages.
- Introduces the House Rent Management System to users.



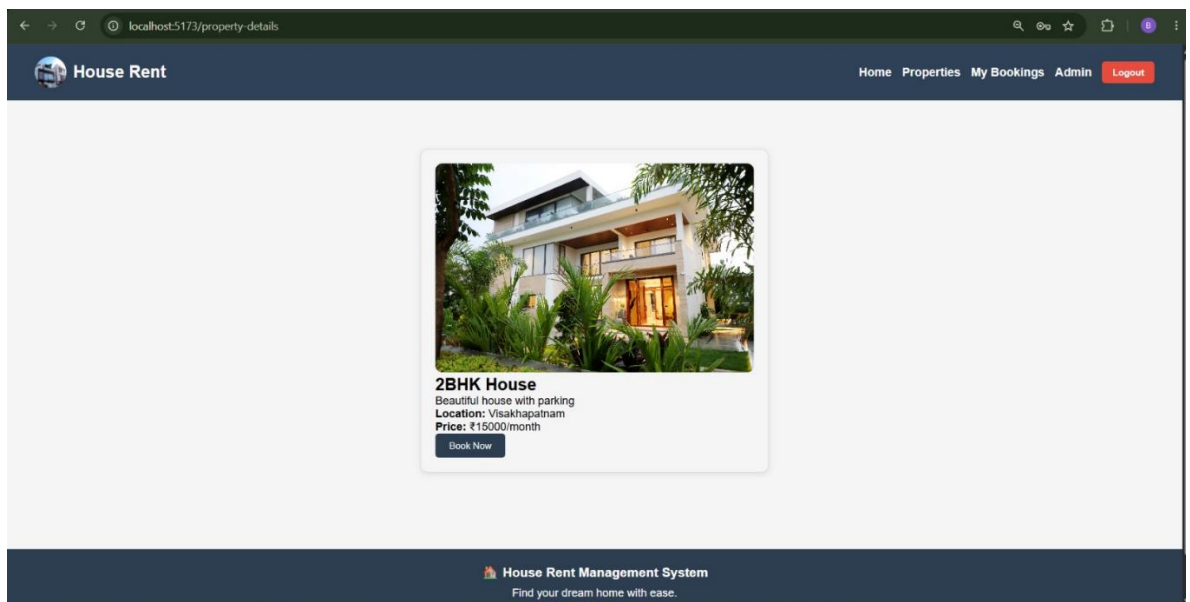
2. Properties Page

- Displays all available rental properties fetched from MongoDB Atlas.
- Shows the property image, title, description, location, and monthly rent.
- Allows users to search properties by title or location.
- Provides **View Details** and **Book Now** options.



3. Property Details Page

- Displays complete information about the selected property.
- Shows property image, description, location, and rental price.
- Helps users view complete property information before booking.



4. User Registration Page

- Allows new users to create an account.
- Collects user details such as Name, Email, and Password.
- Stores the user information securely in the database.

The screenshot shows a web browser at the URL `localhost:5173/register`. The page has a dark blue header with the "House Rent" logo on the left and "Login Register" links on the right. The main content area is light gray and features a "Register" form. The form consists of three input fields: "Enter Name", "Enter Email", and "Enter Password", each with a light blue border. Below these fields is a dark blue "Register" button. The footer is dark blue and contains the text "House Rent Management System", "Find your dream home with ease.", and "© 2020 All Rights Reserved".

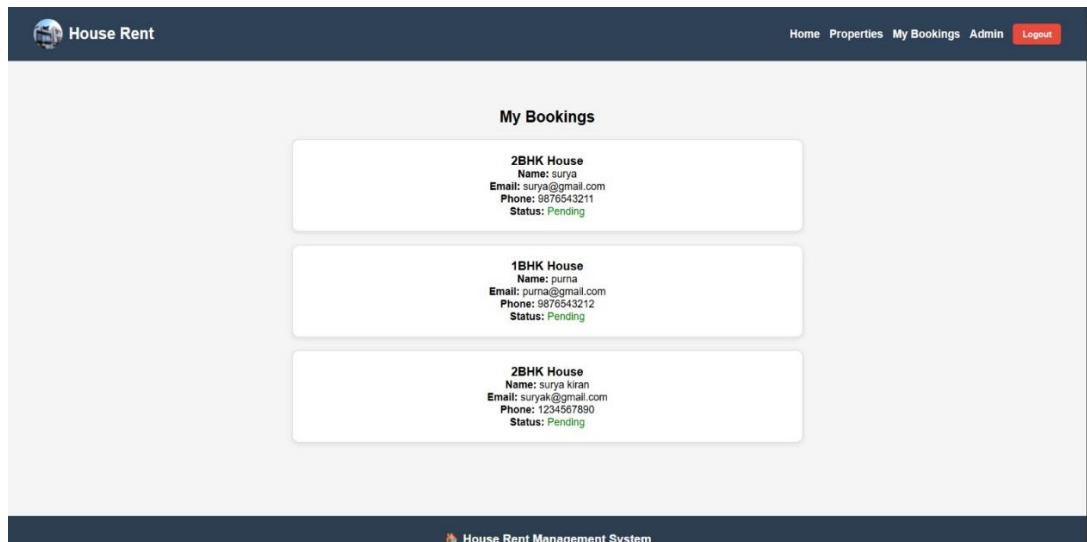
5. Login Page

- Allows registered users to log into the system securely.
- Authenticates users using JWT.
- Redirects users to the application after successful login.

The screenshot shows a web browser at the URL `localhost:5173/login`. The page has a dark blue header with the "House Rent" logo on the left and "Login Register" links on the right. The main content area is light gray and features a "Login" form. The form consists of two input fields: "Enter Email" and "Enter Password", each with a light blue border. Below these fields is a dark blue "Login" button. The footer is dark blue and contains the text "House Rent Management System", "Find your dream home with ease.", and "© 2020 All Rights Reserved".

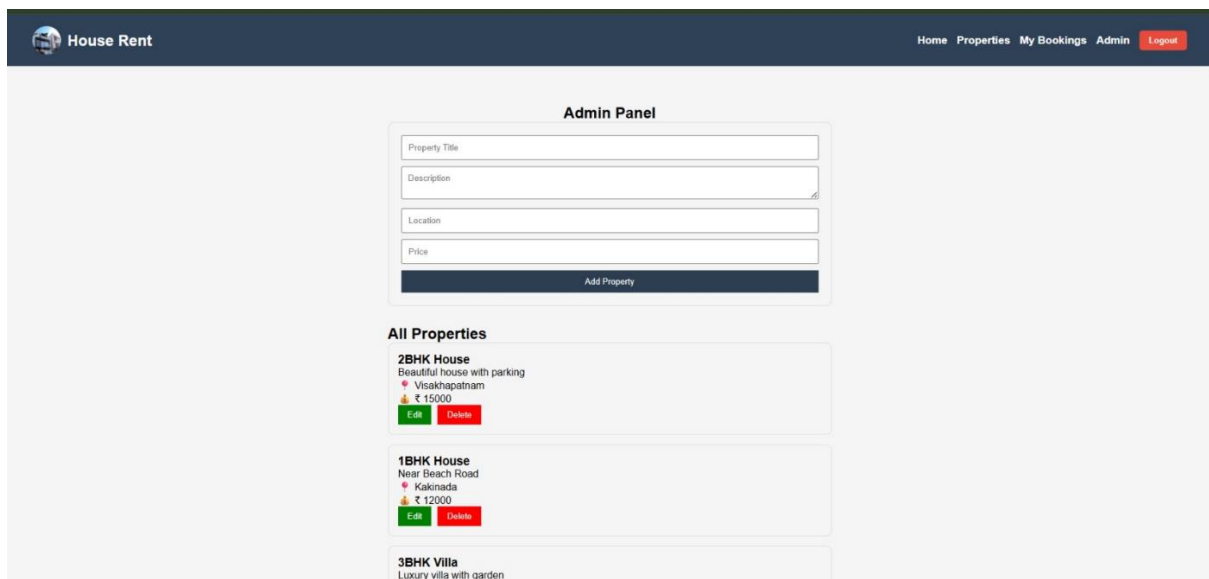
6. My Bookings Page

- Allows users to book their selected rental property.
- Displays the details of booked properties.
- Enables users to view their booking history after successful booking.
- Stores and retrieves booking information securely from MongoDB Atlas.



7. Admin Dashboard

- Allows the administrator to manage rental properties.
- Supports adding, updating, and deleting property details.
- Helps maintain the list of available rental properties.



10. Testing

The House Rent Management System was thoroughly tested to ensure that all modules function correctly and provide a smooth user experience.

Testing Strategy

The following testing activities were performed during development:

- Verified user registration and login functionality.
- Tested JWT-based authentication and authorization.
- Verified password encryption using **bcrypt.js**.
- Tested all REST API endpoints.
- Verified CRUD operations for property management.

- Tested property booking functionality.
- Verified booking history for logged-in users.
- Confirmed database operations using MongoDB Atlas.
- Tested the responsive user interface on different screen sizes.
- Validated invalid inputs and error handling.

Testing Tools Used

Tool	Purpose
Thunder Client (VS Code)	API testing (GET, POST, PUT, DELETE requests)
MongoDB Atlas	Database verification and record validation
Google Chrome	Frontend testing and responsive UI testing
React Developer Tools	Component inspection and debugging
Node.js Console	Backend debugging and server logs
Visual Studio Code	Development and debugging

Test Cases Performed

- User Registration
- User Login
- JWT Token Generation
- JWT Token Verification
- Add Property
- View Properties
- Search Properties
- View Property Details
- Book Property
- View My Bookings
- Update Property
- Delete Property
- Logout Functionality
- Database Record Verification

Testing Result

All functionalities of the House Rent Management System were successfully tested. The frontend and backend communicated correctly through REST APIs, user authentication worked securely using JWT, property and booking information was stored and retrieved successfully from MongoDB Atlas, and the application performed as expected without major errors.

11. Screenshots or Demo

Project Demo Video Link:

https://drive.google.com/file/d/1CaFrYDtIMynhrO9hFoUeIg84isyvxi4X/view?usp=drive_link

12. Known Issues

Although the **House Rent Management System** functions correctly, there are a few limitations that can be improved in future versions.

Known Issues

- Online payment integration is not available.
- Users cannot upload or change their profile picture.
- Email verification and password recovery features are not implemented.
- Property images are currently managed manually and can be improved with image upload functionality.
- Advanced search filters such as property type, price range, and number of bedrooms are not available.
- The application has been tested mainly on modern web browsers; additional testing on different browsers and mobile devices can further improve compatibility.
- Performance optimization techniques such as image optimization and lazy loading can be implemented in future versions.

13. Future Enhancements

The current House Rent Management System provides the essential features required for online house rental management. However, several additional features can be implemented in future versions to improve usability, security, and scalability.

Future Enhancements

- **Online Payment Integration:** Integrate Razorpay, Stripe, or PayPal for secure online rent payments.
- **Profile Management:** Allow users to update profile details, change passwords, and upload profile pictures.
- **Forgot Password & Email Verification:** Implement email verification during registration and password recovery through email.
- **Advanced Property Search:** Add filters based on location, rent, property type, number of bedrooms, and amenities.
- **Property Image Upload:** Allow administrators to upload multiple images for each property.
- **Property Reviews & Ratings:** Enable users to provide ratings and reviews for rented properties.
- **Notification System:** Send email or SMS notifications for registration, booking confirmation, and booking updates.
- **Admin Analytics Dashboard:** Display property statistics, booking reports, and user information through graphical dashboards.
- **Responsive Mobile Application:** Develop Android and iOS applications using React Native or Flutter.
- **Cloud Deployment:** Deploy the application on platforms such as AWS, Microsoft Azure, or Google Cloud Platform.
- **Performance Optimization:** Improve application performance using caching, lazy loading, and database indexing.

Summary

The House Rent Management System successfully demonstrates the implementation of a full-stack web application using the MERN Stack. The project provides a secure and user-friendly platform where users can register, log in, browse rental properties, search houses, view property details, and book houses efficiently.

The application follows a modular architecture, uses JWT for secure authentication, bcrypt.js for password encryption, and MongoDB Atlas for cloud-based data storage. By implementing the proposed future enhancements, the system can become a more scalable, secure, and feature-rich rental management platform that offers an improved experience for both users and administrators.

